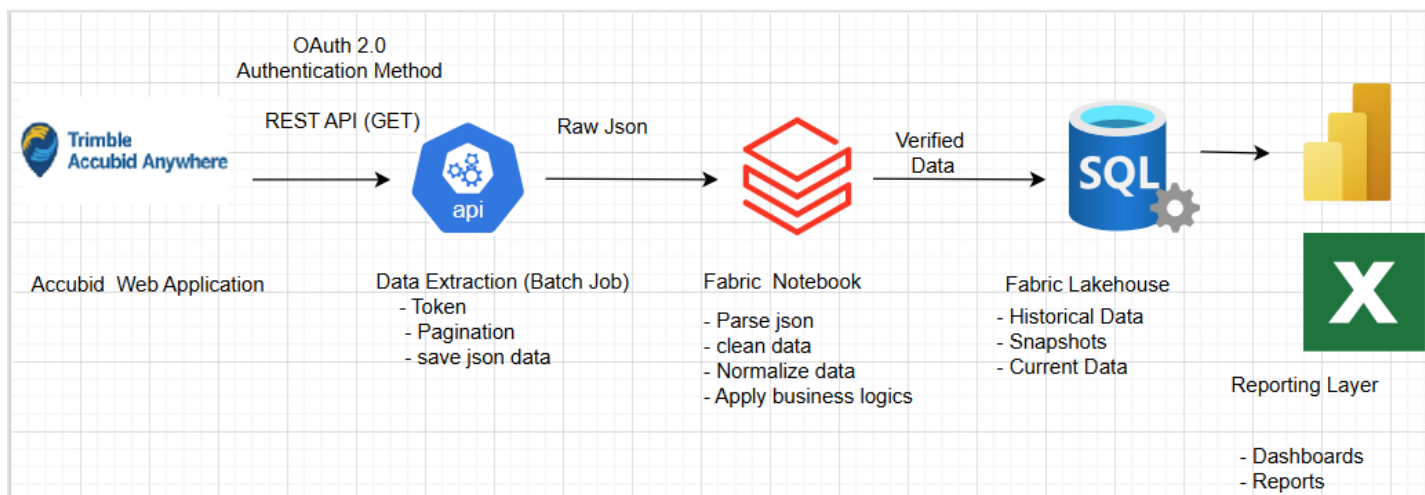


Extracting data from Accubid

Stakeholders: Business Analysts, Preconstruction Department, leadership team members

Purpose: The main purpose of this project is to address some of the key problems the construction project faces during the preconstruction and during the construction phase of the project. A construction project goes through several phases in its lifecycle. Handling the materials, creating an estimate quote, billing, purchasing, finalizing the quote are all the important steps in the project life cycle. All of these steps rely on data, hence it is very important that the stakeholders such as electrical estimators, senior estimators, estimating managers, project managers, project owners have all the accurate and up to date information to make the decision.

High-level Architecture Diagram of the Technical Workflow



Technical Summary:

Cadence: This data architecture pipeline will run every hour during the office hours through 8:00 A:M – 5:00 P:M EST in the Weekdays.

Accubid Web Application: Accubid is a cloud based estimating solution that is used for electrical, ICT and industrial, mechanical contracting works in a construction project. This software provides the complete workflow for the construction projects all the way from estimating up to the purchasing, project management and billing for the clients. It streamlines the entire bidding process and project management process, helping the stakeholders create accurate cost estimates, manage materials, track labors and handle the project billings.

API: To extract data using API, Accubid Trimble’s official documentation has been used in the entire project. To obtain the bearer token for the authentication purposes, Grant Type “Authorization Code” workflow needs to follow. The Authorization Code workflow setup and process is shown in the documentation [Link](#)

Authorization:

Request: The first request is a GET or redirect to the authorization endpoint (/authorize) performed in a web browser. The /authorize endpoint supports three response_types: code, id_token, token. If code is present in your parameters, JWTs are not needed for authorization.

Credentials

This documentation refers to credentials. Credentials are Base64 encoding your combined Client id and Client secret separated by a colon, e.g.: Base64(id:secret).

```
GET /oauth/authorize
```

Response:

```
http://localhost:5173/auth/callback?
code=us_9fd5fee3f3c74267910f8ed7c71c85a8&
state=Hn4K-n1m00000CiUUV-v0UNc0JZ8Jh_4shoo
```

Token:

```
POST /oauth/token
```

Headers

Header	Description
Authorization	Basic <code>credentials</code>
Accept	application/json
Content-Type	application/x-www-form-urlencoded

Body

Body	Description
grant_type	authorization_code
code	The authorization code returned from the above call
client_id	Application ID registered for the application
redirect_uri	Callback URL registered with Identity

Response

If the token request succeeds, the server responds with a 200 response code.

```
{
  "access_token": "8aca1...",
  "refresh_token": "861e9...",
  "expires_in": 3600,
  "id_token": "eyJhbG..."
}
```

Refresh

If the access token expires, it can be refreshed using the one-time refresh token.

The server will respond with a Token Response.

POST /oauth/token

Get database token

GET https://cloud.api.trimble.com/anywhere/database/v1/databases

This endpoint will return the list of database names and tokens.

Response:

```
[
  {
    "Token": "xxxxxxxxxx",
    "DatabaseName": "Database A",
    "CompanyName": "Company A"
  },
  {
    "Token": "yyyyyyyyyy",
    "DatabaseName": "Database B",
    "CompanyName": "Company B"
  }
]
```

Get project list

Get the project list for a database

GET https://cloud.api.trimble.com/anywhere/project/v1/projects/{databaseToken}

This will return a list of project names and IDs.

Response:

```
[
  {
    "ProjectID": "XxYyZz123",
    "ProjectName": "Project A",
    "ProjectNumber": "Project 1",
    "Type": "Commercial:Large:New",
    "StartDate": "1970-01-01T00:00:00.000Z",
    "EndDate": "1970-01-01T00:00:00.000Z",
    "CreatedDate": "1970-01-01T00:00:00.000Z",
    "CreatedBy": "exampleresponse@trimble.com",
    "ManagingBranchName": "Company Name",
    "ProjectPath": "Project Folder",
    "TrimbleProjectID": null
  },
  {
    "ProjectID": "XxYyZz123",
    "ProjectName": "Project B",
    "ProjectNumber": "Project 2",
    "Type": "Commercial:Large:New",
    "StartDate": "1970-01-01T00:00:00.000Z",
    "EndDate": "1970-01-01T00:00:00.000Z",
    "CreatedDate": "1970-01-01T00:00:00.000Z",
    "CreatedBy": "exampleresponse@trimble.com",
    "ManagingBranchName": "Company Name",
    "ProjectPath": "Project Folder",
    "TrimbleProjectID": null
  }
]
```

Get project info

Get information for a specific project

```
GET
https://cloud.api.trimble.com/anywhere/project/v1/project/{databaseToken}/{projectID}
```

Response:

```
[
  {
```

```

    "ProjectID": "XxYyZz123",
    "ProjectPath": "Project Folder",
    "ProjectName": "Project A",
    "ProjectNumber": "Project 1",
    "Type": "Commercial:Large:New",
    "StartDate": "1970-01-01T00:00:00.000Z",
    "EndDate": "1970-01-01T00:00:00.000Z",
    "CreatedDate": "1970-01-01T00:00:00.000Z",
    "CreatedBy": "accubid",
    "ManagingBranchName": "Company Name",
    "SiteName": null,
    "SitePhone": null,
    "SiteFax": null,
    "SiteStreet1": null,
    "SiteStreet2": null,
    "SiteCity": null,
    "SiteState": null,
    "SiteCountry": null,
    "SiteZipCode": null,
    "TrimbleProjectID": null,
    "FolderID": "XxYyZz456",
    "Vendors": []
  }
]

```

Get estimate list

Get the estimate list for a project

```

GET
https://cloud.api.trimble.com/anywhere/estimate/v1/estimates/{databaseToken}/{projectID}

```

This will return a list of estimated names and IDs

Response:

```

[
  {
    "EstimateID": "AaBbCc123",
    "EstimateName": "Estimate A",
    "EstimateNumber": "Estimate 1",
    "Status": "Pending",
    "CreatedBy": "exampleresponse@trimble.com",
  }
]

```

```

    "CreateDate": "1970-01-01T00:00:00.000Z",
    "DueDate": "1970-01-01T00:00:00.000Z",
    "StartDate": "1970-01-01T00:00:00.000Z",
    "EndDate": "1970-01-01T00:00:00.000Z",
    "Contract": null,
    "IsDeleted": false
  },
  {
    "EstimateID": "XxYyZz456",
    "EstimateName": "Estimate B",
    "EstimateNumber": "Estimate 2",
    "Status": "Pending",
    "CreatedBy": "exampleresponse@trimble.com",
    "CreateDate": "1970-01-01T00:00:00.000Z",
    "DueDate": "1970-01-01T00:00:00.000Z",
    "StartDate": "1970-01-01T00:00:00.000Z",
    "EndDate": "1970-01-01T00:00:00.000Z",
    "Contract": null,
    "IsDeleted": false
  }
]

```

Get estimate info

Get information for a specific estimate

```

GET
https://cloud.api.trimble.com/anywhere/estimate/v1/estimate/{databaseToken}/{estimateID}

```

This will return information for a specific estimate

Response:

```

{
  "EstimateID": "AaBbCc123",
  "Name": "Estimate A",
  "Number": "Estimate 1",
  "CreatedBy": "exampleresponse",
  "CreateDate": "2015-06-03T13:42:06.817",
  "LaborColumn": "Column Nmae",
  "LaborFactorMethod": "Additive",
  "ImperialMetric": "Imperial",
  "Status": "Pending",
  "Industry": "Mechanical",

```

```
"ApplyMarkupOnOverhead": true,
"ApplyOverheadAndMarkupOnTax": true,
"VendorPricingPrecedence": true,
"Contract": null,
"DueDate": "1970-01-01T00:00:00.000Z",
"StartDate": "1970-01-01T00:00:00.000Z",
"EndDate": "1970-01-01T00:00:00.000Z",
"Notes": "<div style=\"font-size:12pt;font-family:"Times New
Roman";\"></div>",
"Duration": 0.0,
"TimeInterval": "Days",
"IsDeleted": false,
"BidSummaries": [
  {
    "BidSummaryID": "xxxxyyyy",
    "BidSummaryName": "Bidsummary A"
  },
  {
    "BidSummaryID": "aaaabbbb",
    "BidSummaryName": "Bidsummary B"
  }
],
"BreakdownCategories": [
  {
    "BreakdownCategoryID": 1,
    "BreakdownCategoryName": "Area",
    "BreakdownCategoryLabel": null
  },
  {
    "BreakdownCategoryID": 2,
    "BreakdownCategoryName": "Bid Item",
    "BreakdownCategoryLabel": null
  },
  {
    "BreakdownCategoryID": 3,
    "BreakdownCategoryName": "Misc 1",
    "BreakdownCategoryLabel": "Line Number"
  },
  {
    "BreakdownCategoryID": 4,
    "BreakdownCategoryName": "Drawing",
    "BreakdownCategoryLabel": null
  }
]
}
```

Get Bid Breakdown View list

Call the Bid Breakdown View API to get the list of Bid Breakdown Views for an Estimate

GET

```
https://cloud.api.trimble.com/anywhere/closeout/v1/bidbreakdownview/{databaseToken}/{estimateID}
```

This will return the list of Bid Breakdown Views for a specific Estimate.

Response:

```
[
  {
    "BidBreakdownViewId": "abcd",
    "Name": "SC1",
    "Order": 0,
    "GroupOptions": [
      {
        "Id": "5",
        "Name": "Bid Item",
        "Order": 1
      },
      {
        "Id": "7",
        "Name": "Sort Code 1",
        "Order": 2
      }
    ],
    "AutoAllocateRemainderIncidentalLabor": true,
    "AutoAllocateRemainderLaborFactoring": true,
    "AutoAllocateRemainderLaborEscalation": true,
    "AutoAllocateRemainderIndirectLabor": true,
    "AutoAllocateRemainderSubcontracts": true,
    "AutoAllocateRemainderEquipment": true,
    "AutoAllocateRemainderGeneralExpenses": true,
    "AutoAllocateRemainderQuotes": true,
    "RoundTotal": 0,
    "AutoAllocationMethod": 1
  },
  {
    "BidBreakdownViewId": "efghjsd",
    "Name": "SC2",
```

```

"Order": 1,
"GroupOptions": [
  {
    "Id": "1",
    "Name": "Drawing",
    "Order": 1
  },
  {
    "Id": "8",
    "Name": "Sort Code 2",
    "Order": 2
  }
],
"AutoAllocateRemainderIncidentalLabor": true,
"AutoAllocateRemainderLaborFactoring": true,
"AutoAllocateRemainderLaborEscalation": true,
"AutoAllocateRemainderIndirectLabor": true,
"AutoAllocateRemainderSubcontracts": true,
"AutoAllocateRemainderEquipment": true,
"AutoAllocateRemainderGeneralExpenses": true,
"AutoAllocateRemainderQuotes": true,
"RoundTotal": 0,
"AutoAllocationMethod": 1
}
]

```

Get Bid Breakdown Details

Call the Bid Breakdown API to get the details of Bid Breakdown for a Bid Summary and Bid Breakdown View

```

GET
https://cloud.api.trimble.com/anywhere/closeout/v1/bidbreakdown/{databaseToken}/{bidSummaryID}/{bidBreakdownViewID}?pageIndex={pageNumber}

```

This will return the paginated Bid Breakdown Details for specific Bid Summary and Bid Breakdown View.

Response:

```

{
  "pageIndex": 0,
  "totalItems": 2,

```

```
"pageSize": 100,
"items": [
  {
    "BidBreakdownViewId": "abcd",
    "BidSummaryId": "efgh",
    "GroupedCategories": [
      {
        "Description": "No Bid Item",
        "Id": "xyz",
        "Order": 1
      },
      {
        "Description": "",
        "Id": "",
        "Order": -1
      }
    ],
    "RowType": "Regular",
    "TotalAmount": 136,
    "TotalMaterialAmount": 36,
    "TotalLaborHours": 10,
    "TotalLaborAmount": 100,
    "FieldLaborHours": 10,
    "FieldLaborAmount": 100,
    "ShopLaborHours": 0,
    "ShopLaborAmount": 0,
    "IncidentalLaborHours": 0,
    "IncidentalLaborAmount": 0,
    "LaborFactoringHours": 0,
    "LaborFactoringAmount": 0,
    "LaborEscalationAmount": 0,
    "IndirectLaborHours": 0,
    "IndirectLaborAmount": 0,
    "Equipment": 0,
    "GeneralExpenses": 0,
    "Subcontracts": 0,
    "Quotes": 0,
    "Overhead": 0,
    "MarkUp": 0,
    "DefineAdj": 0,
    "RoundingAdj": 0,
    "Unit": 0
  },
  {
    "BidBreakdownViewId": "abcd",
```

```
"BidSummaryId": "efgh",
"GroupedCategories": [
  {
    "Description": "No Bid Item",
    "Id": "qwerty",
    "Order": 1
  },
  {
    "Description": "Common Tasks",
    "Id": "asdfg",
    "Order": 1002
  }
],
"RowType": "Regular",
"TotalAmount": 136.67,
"TotalMaterialAmount": 91.67,
"TotalLaborHours": 4.5,
"TotalLaborAmount": 45,
"FieldLaborHours": 4.5,
"FieldLaborAmount": 45,
"ShopLaborHours": 0,
"ShopLaborAmount": 0,
"IncidentalLaborHours": 0,
"IncidentalLaborAmount": 0,
"LaborFactoringHours": 0,
"LaborFactoringAmount": 0,
"LaborEscalationAmount": 0,
"IndirectLaborHours": 0,
"IndirectLaborAmount": 0,
"Equipment": 0,
"GeneralExpenses": 0,
"Subcontracts": 0,
"Quotes": 0,
"Overhead": 0,
"MarkUp": 0,
"DefineAdj": 0,
"RoundingAdj": 0,
"Unit": 0
},
"links": {
  "self":
"https://cloud.api.trimble.com/anywhere/closeout/v1/bidbreakdown/databaseToken/bidSummaryId/bidBreakdownviewId?pageIndex=1",

```

```

    "first":
    "https://cloud.api.trimble.com/anywhere/closeout/v1/bidbreakdown/databaseToken/bi
dSummaryId/bidBreakdownviewId?pageIndex=0",
    "last":
    "https://cloud.api.trimble.com/anywhere/closeout/v1/bidbreakdown/databaseToken/bi
dSummaryId/bidBreakdownviewId?pageIndex=2"
  }
}

```

Fabric Notebook:

Fabric Notebook is used to read the json data that is obtained from the web api using the GET method. In the Fabric notebook various python and pyspark codes are used to convert the json into tabular data, clean the data, apply business logic and then validate the data. Once the data is validated then it is loaded into the Fabric Data Lakehouse

Sample code using DUMMY DATA SETS

```

import os
import requests
import json

# -----
# CONFIGURATION ( Key Vault Secrets )
# -----
TRIMBLE_ACCESS_TOKEN = os.getenv("TRIMBLE_ACCESS_TOKEN", "DUMMY_TRIMBLE_TOKEN")

BASE_URL = "https://cloud.api.trimble.com/anywhere"
HEADERS = {
    "Authorization": f"Bearer {TRIMBLE_ACCESS_TOKEN}",
    "Accept": "application/json"
}

# -----
# STEP 1: GET DATABASE TOKENS
# -----
def get_database_tokens():
    url = f"{BASE_URL}/database/v1/databases"
    response = requests.get(url, headers=HEADERS)
    response.raise_for_status()
    return response.json()

# -----

```

```

# STEP 2: GET PROJECT LIST FOR A DATABASE
# -----
def get_projects(database_token):
    url = f"{BASE_URL}/project/v1/projects/{database_token}"
    response = requests.get(url, headers=HEADERS)
    response.raise_for_status()
    return response.json()

def main():
    databases = get_database_tokens()

    # Dummy selection: use the first database
    db = databases[0]
    database_token = db["Token"]

    projects = get_projects(database_token)

    # Write raw data to Fabric Lakehouse Files area
    raw_path = "/lakehouse/default/Files/accubid/raw/projects.json"
    with open(raw_path, "w") as f:
        json.dump(projects, f)

    print(f" Extracted {len(projects)} projects")
    print(f" Raw data written to {raw_path}")

if __name__ == "__main__":
    main()

from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, upper, to_date, when,
    year, current_timestamp
)
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

spark = SparkSession.builder.getOrCreate()

# -----
# READ RAW DATA FROM FABRIC LAKEHOUSE
# -----
raw_path = "Files/accubid/raw/projects.json"

df_raw = spark.read.json(raw_path)

```

```

# -----
# DATA CLEANING
# -----
df_clean = (
    df_raw
    .withColumn("ProjectName", upper(col("ProjectName")))
    .withColumn("ProjectType", upper(col("Type")))
    .withColumn("StartDate", to_date(col("StartDate")))
    .withColumn("EndDate", to_date(col("EndDate")))
    .dropDuplicates(["ProjectID"])
)

# -----
# BUSINESS LOGIC (ESTIMATING)
# -----
df_logic = (
    df_clean
    .withColumn(
        "ProjectDurationCategory",
        when(col("EndDate").isNull(), "ONGOING")
        .when(year(col("EndDate")) - year(col("StartDate")) > 2, "LONG_TERM")
        .otherwise("SHORT_TERM")
    )
    .withColumn(
        "MarketSegment",
        when(col("ProjectType").like("%COMMERCIAL%"), "COMMERCIAL")
        .when(col("ProjectType").like("%INDUSTRIAL%"), "INDUSTRIAL")
        .otherwise("OTHER")
    )
    .withColumn("etl_loaded_at", current_timestamp())
)

# -----
# DEDUP USING LATEST RECORD LOGIC
# -----
window_spec =
Window.partitionBy("ProjectID").orderBy(col("etl_loaded_at").desc())

df_final = (
    df_logic
    .withColumn("rn", row_number().over(window_spec))
    .filter(col("rn") == 1)
    .drop("rn")
)

```

```

# -----
# WRITE TO FABRIC LAKEHOUSE
# -----
df_final.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("accubid_projects_curated")

print(" Data successfully written to Fabric Lakehouse table:
accubid_projects_curated")

```

```

import os
import json
import requests

# -----
# CONFIGURATION
# -----
TRIMBLE_ACCESS_TOKEN = os.getenv("TRIMBLE_ACCESS_TOKEN", "DUMMY_TRIMBLE_TOKEN")
BASE_URL = "https://cloud.api.trimble.com/anywhere"

HEADERS = {
    "Authorization": f"Bearer {TRIMBLE_ACCESS_TOKEN}",
    "Accept": "application/json"
}

# Location where projects were written previously
PROJECTS_PATH = "/lakehouse/default/Files/accubid/raw/projects.json"

# -----
# GET ESTIMATES FOR A PROJECT
# -----
def get_estimates(database_token: str, project_id: str):
    """
    Retrieve list of estimates for a given ProjectID and DatabaseToken.
    """
    url = (
        f"{BASE_URL}/estimate/v1/estimates/"
        f"{database_token}/{project_id}"
    )
    response = requests.get(url, headers=HEADERS)

```

```

        response.raise_for_status()
        return response.json()

def main():
    # Load previously extracted projects
    with open(PROJECTS_PATH, "r") as f:
        projects = json.load(f)

    # (tokens are stable and can be reused)
    database_token = os.getenv("ACCUBID_DATABASE_TOKEN", "DUMMY_DB_TOKEN")

    all_estimates = []

    for project in projects:
        project_id = project["ProjectID"]

        estimates = get_estimates(database_token, project_id)

        for e in estimates:
            e["ProjectID"] = project_id
            e["DatabaseToken"] = database_token
            all_estimates.append(e)

    # Write raw estimate data to Fabric Lakehouse
    estimates_path = "/lakehouse/default/Files/accubid/raw/estimates.json"
    with open(estimates_path, "w") as f:
        json.dump(all_estimates, f)

    print(f" Extracted {len(all_estimates)} estimates")
    print(f" Raw estimates written to {estimates_path}")

if __name__ == "__main__":
    main()

```

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col, upper, to_date, when, current_timestamp
)
from pyspark.sql.window import Window

```

```

from pyspark.sql.functions import row_number

spark = SparkSession.builder.getOrCreate()

# -----
# READ RAW ESTIMATES
# -----
raw_estimates_path = "Files/accubid/raw/estimates.json"
df_raw = spark.read.json(raw_estimates_path)

# -----
# DATA CLEANING
#-----
df_clean = (
    df_raw
    .withColumn("EstimateName", upper(col("EstimateName")))
    .withColumn("EstimateStatus", upper(col("Status")))
    .withColumn("CreatedDate", to_date(col("CreatedDate")))
    .withColumn("ModifiedDate", to_date(col("ModifiedDate")))
    .dropDuplicates(["EstimateID"])
)

# -----
# BUSINESS LOGIC (ESTIMATING)
# -----
df_logic = (
    df_clean
    .withColumn(
        "BidLifecycleStage",
        when(col("EstimateStatus") == "FINAL", "AWARDED/CLOSED")
        .when(col("EstimateStatus") == "IN_PROGRESS", "ACTIVE BID")
        .otherwise("ARCHIVED")
    )
    .withColumn(
        "IsActiveBid",
        when(col("EstimateStatus").isin("IN_PROGRESS", "DRAFT"), True)
        .otherwise(False)
    )
    .withColumn("etl_loaded_at", current_timestamp())
)

# -----
# KEEP LATEST VERSION PER ESTIMATE
# -----
window_spec = (

```

```

Window
  .partitionBy("EstimateID")
  .orderBy(col("ModifiedDate").desc())
)

df_final = (
  df_logic
  .withColumn("rn", row_number().over(window_spec))
  .filter(col("rn") == 1)
  .drop("rn")
)

# -----
# WRITE TO FABRIC LAKEHOUSE
# -----
df_final.write \
  .format("delta") \
  .mode("overwrite") \
  .saveAsTable("accubid_estimates_curated")

print("Estimates successfully written to accubid_estimates_curated")

```

Datawarehouse: All the verified data is now loaded in the Fabric Lakehouse. Fabric Lakehouse works as a Datawarehouse where all the historical, current and snapshot data is stored for all the reporting purposes.

Reporting Layer: Reporting tools such as Power BI and Microsoft Excel are used to create reports and dashboards using the data stored in the tables of the Fabric Lakehouse.

References:

- <https://developer.trimble.com/docs/authentication/guides/authorization-code>
- <https://developer.trimble.com/docs/accubid-anywhere/guides/access/>
- <https://developer.trimble.com/docs/accubid-anywhere/tools/api/database/>
- <https://developer.trimble.com/docs/accubid-anywhere/tools/api/project/>
- <https://developer.trimble.com/docs/accubid-anywhere/tools/api/estimate/>